

Universidad Nacional Abierta Y A Distancia

Actividad:

Informe Académico De Actividad Fase 5

Nombre actividad:

Evaluación POA FPD

Unidad Gestora:

Escuela De Ciencias Básicas Tecnología E Ingeniería Ecbti

Programa:

Ingeniería De Sistemas

Curso: **Fundamentos De Programación**

Código: **213022**

Alumno:

Dominguez Gallardo Leon

Grupo: **213022A_2202_80**

CONTENIDO

2. Descripción del problema a desarrollar.....	3
3. Análisis de Información de la situación problemática.....	4
4. Tabla de requerimientos.....	5
5. Diagrama De Flujo Propuesto:.....	6
6. Ejecución y Desarrollo.....	8
Repositorio de evidencias:.....	8
Implementación de repositorio:.....	8
7. conclusión.....	12
8. Referencias.....	13

1. Introducción

La programación es una disciplina esencial en el desarrollo de sistemas informáticos, ya que permite estructurar soluciones a problemas complejos mediante el diseño de algoritmos, el uso de funciones y la aplicación de estructuras de control. El aprendizaje de estos conceptos no solo fortalece las competencias técnicas, sino que también fomenta el pensamiento lógico y la capacidad de abstracción, elementos clave en la formación de profesionales en ingeniería y ciencias de la computación.

El estudio de lenguajes como C y C++ ha sido ampliamente documentado en la literatura especializada. **Ceballos Sierra (2015)** expone en *C/C++. Curso de programación* los fundamentos de operadores y estructuras de datos, aportando bases sólidas para comprender la construcción de expresiones y la organización de programas. Por su parte, **Deitel y Deitel (2016)**, en *Cómo programar en C/C++*, profundizan en la definición y uso de funciones, destacando la importancia de la modularización en el desarrollo de algoritmos.

Asimismo, el trabajo colaborativo en entornos de programación requiere herramientas que faciliten la gestión de versiones y la cooperación entre desarrolladores. En este sentido, **GitHub, Inc. (2025)**, en *Acerca de GitHub y Git*, presenta conceptos y buenas prácticas que permiten identificar los elementos básicos del flujo de trabajo colaborativo, fortaleciendo la calidad y eficiencia en proyectos de software.

En conjunto, estas referencias bibliográficas proporcionan el marco teórico necesario para comprender los principios de la programación estructurada y las buenas prácticas en el desarrollo de aplicaciones, constituyendo la base para abordar ejercicios prácticos orientados al fortalecimiento de las competencias en programación.

2. Descripción del problema a desarrollar

El problema elegido es:

Problema 3: Se requiere una herramienta para auditar el inventario y decidir qué artículos necesitan ser reabastecidos. La información se encuentra en una matriz: [Código Artículo, Nombre, Stock Actual, Stock Mínimo Requerido].

Requisitos de Desarrollo

- Matriz: Crear una matriz con al menos 5 artículos.
- Módulos: Se requiere un módulo (función) para determinar la cantidad exacta a pedir para un artículo.

- Lógica de Negocio:

- ✓ Si el Stock Actual es menor al Stock Mínimo, la cantidad a pedir es la diferencia (Mínimo Requerido - Stock Actual).
- ✓ Si el Stock Actual es suficiente (mayor o igual al Mínimo),

la cantidad a pedir es cero.

- Salida: Imprimir una lista de pedidos que muestre el nombre del artículo y la cantidad exacta que debe ser solicitada.

3. Análisis de Información de la situación problemática

La gestión de inventarios es un proceso crítico en cualquier organización, ya que garantiza la disponibilidad de productos y evita pérdidas económicas derivadas de faltantes o excesos de stock. En la situación planteada, se evidencia la necesidad de contar con una herramienta que permita **auditar de manera sistemática el inventario** y determinar qué artículos requieren reabastecimiento.

El problema surge porque, sin un control automatizado, la identificación de productos en déficit depende de revisiones manuales, lo que incrementa el riesgo de errores y retrasa la toma de decisiones. Además, la ausencia de validaciones y mecanismos de retroalimentación puede generar inconsistencias en el registro de unidades, afectando la confiabilidad de la información.

La problemática, por tanto, se centra en la necesidad de implementar un sistema que integre **funciones de consulta, actualización y validación**, con el fin de optimizar el control de inventarios y asegurar la continuidad de las operaciones. Este análisis evidencia que la solución debe ser **interactiva, robusta y tolerante a errores**, para responder de manera efectiva a las exigencias de la gestión de stock en escenarios reales.

4. Tabla de requerimientos

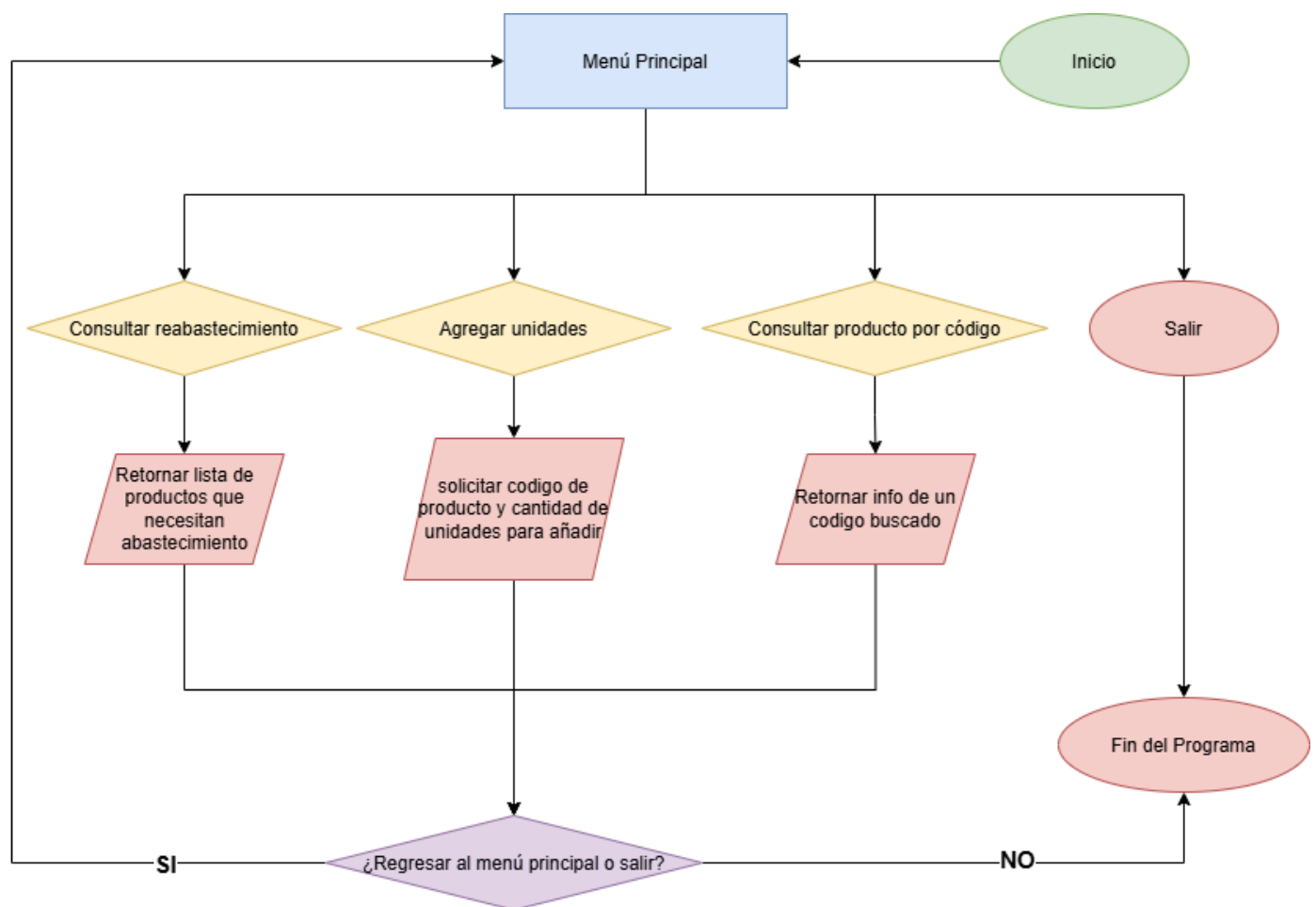
Como parte del análisis inicial de un problema que requiere abordarse, desarrollarse y resolverse dentro del ámbito de la programación en sistemas estructurados, es necesario generar una tabla de requerimientos como parte del proceso del análisis.

ID	Descripción	Entradas	Resultados (salidas)
R1	Definir matriz inicial con al menos 5 artículos [Código, Nombre, Stock Actual, Stock Mínimo].	Ninguna	Matriz almacenada en memoria.
R2	Mostrar menú principal con opciones (1–4).	Ninguna	Menú en pantalla.
R3	Validar opción ingresada por el usuario (solo valores 1–4).	Opción ingresada	Mensaje de error si es inválida, continuar si es válida.
R4	Consultar productos que necesitan reabastecimiento.	Matriz de artículos	Lista de productos con cantidad a pedir.
R5	Agregar unidades a un producto existente.	Código del producto y cantidad	Actualización del stock del producto.
R6	Validar que la cantidad ingresada sea ≥ 0 .	Cantidad ingresada	Mensaje de error si es negativa, actualización si es válida.
R7	Consultar información de un producto por código.	Código del producto	Mostrar datos del producto o mensaje de error si no existe.
R8	Opción de salir desde el menú principal.	Opción 4	Mensaje de finalización del programa.
R9	Submenú interno: preguntar si desea regresar al menú principal o salir.	Entrada del usuario (S/N)	Regreso al menú o finalización del programa.
R10	Validar entrada del submenú (solo S o N).	Entrada del usuario	Mensaje de error si es inválida, repetir pregunta.
R11	Manejo de excepciones en entradas (códigos y cantidades).	Datos ingresados por el usuario	Mensajes claros de error y continuidad del programa.

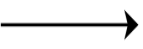
La definición clara de requerimientos es parte esencial del análisis en programación estructurada, pues permite identificar entradas, procesos y salidas de manera ordenada (Ramírez Marín, 2019).

5. Diagrama De Flujo Propuesto:

Como parte de una buena práctica y para comprender cómo se puede abordar el desarrollo del problema, antes de ir a la codificación, Se realiza un diagrama de flujo conceptual que permita entender las operaciones implicadas en problema a desarrollar, ya con la ayuda de este diagrama podemos abordar ampliamente los pasos y regresar por si necesitamos comprender o mejorar algún aspecto.



Se realizó un diagrama de flujo que representa gráficamente el proceso de entrada, cálculo y salida final, siguiendo la metodología de programación estructurada (Moreno Pérez, 2014).

Convenciones de Diagrama			
Símbol o	Figura	Nombre	Función en el Diagrama
Elipse / Óvalo		(Inicio/Fin)	Indica el punto de inicio y el punto de finalización del algoritmo. En el diagrama, tenemos dos: uno que dice Inicio y otro que dice Fin.
Paralel ogramo		Entrada o Salida	Representa cualquier operación de entrada de datos (solicitar información al usuario) o salida de resultados (mostrar un mensaje en pantalla). Ejemplos en este diagrama: Solicitar cantidad de payasos" y "Mostrar peso total.
Rectán gulo		Proceso	Define una operación o un cálculo interno, es decir, una acción que el programa ejecuta pero sin interactuar directamente con el usuario. En este caso, se usa para Calcular peso total y también para Definir las constantes de los muñecos.
Rombo / Diaman te		Decisión o condición	Representa un punto de bifurcación en el algoritmo básicamente es una toma de decisiones. Se utiliza para indicar una pregunta y condición. El flujo del programa se divide en dos caminos (Sí/No o Verdadero/Falso). se interpreta como un if en el código
Flecha / Conect or		Línea de Flujo	Son las líneas que unen todos los demás símbolos. Indican la dirección y la secuencia que sigue el algoritmo. Sin ellas, los pasos no tendrían un orden. Se usan en todo el diagrama para conectar las figuras entre sí, en este caso algunas son punteadas para indicar un flujo especial de repetición o bucle while en el código.

6. Ejecución y Desarrollo

Repositorio de evidencias:

Con la finalidad de cumplir con los lineamientos de la rúbrica evaluativa.

El desarrollo de la parte práctica de esta actividad exige el uso de las siguientes herramientas donde serán publicadas las evidencias y sustentación de la actividad.

- **Repositorio Github publico:**
<https://github.com/leondominguez/unadfase5-audinv#>
- **Video De Presentación publico oculto:** https://youtu.be/0vjES_Ely_4

Implementación de repositorio:

se realiza la creación utilizando gitbash y codificando desde jupyter notebook
repositorio desplegado y funcionando:

```
None

# Clonacion de Proyecto
## Clonar el repositorio desde GitHub (usando SSH)
- git clone git@github.com:leondominguez/unadfase5-audinv.git
- https://github.com/leondominguez/unadfase5-audinv.git

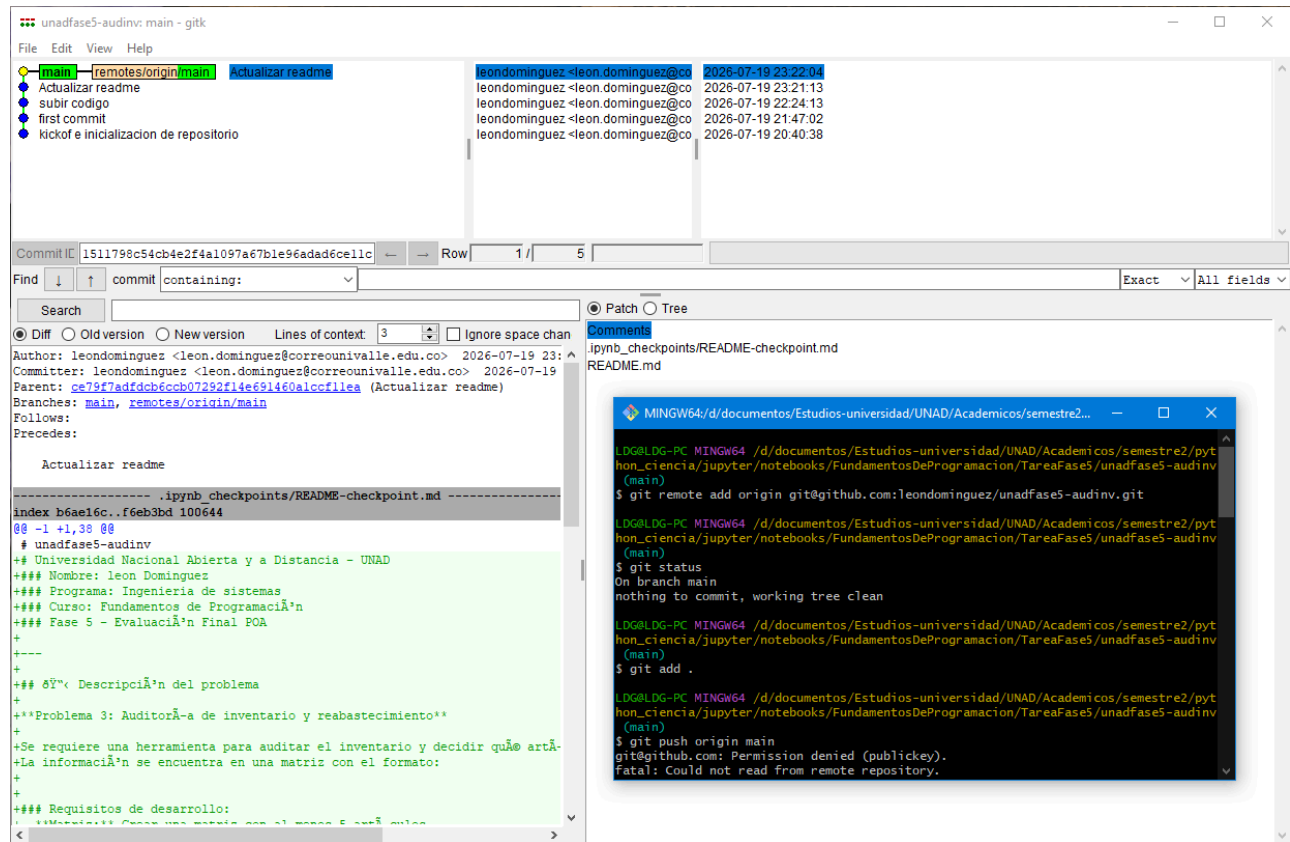
## Entrar al directorio del proyecto
cd unadfase5-audinv

## Verificar el remoto configurado
git remote -v

## Traer cambios del repositorio remoto a tu copia local
git pull origin main

## Subir cambios locales reemplazando el remoto
git push origin main --force
```

Creación de la rama, kickoff de proyecto y primeros push



Luego del despliegue se realiza la codificación de la solución de acuerdo a los requerimientos y diagrama de flujo

Python

```
# Problema 3: Auditoría de inventario y reabastecimiento con menú seguro
# Autor: Leon Domínguez
# Curso: Fundamentos de Programación - Fase 5 Evaluación Final POA

def calcular_pedido(stock_actual, stock_minimo):
    if stock_actual < 0 or stock_minimo < 0:
        raise ValueError("Los valores de stock deben ser enteros positivos.")
    if stock_actual < stock_minimo:
        return stock_minimo - stock_actual
    else:
        return 0
```

```

def consultar_reabastecimiento(inventario):
    print("\n=== PRODUCTOS QUE NECESITAN REABASTECIMIENTO ===")
    for codigo, nombre, stock_actual, stock_minimo in inventario:
        try:
            cantidad_pedir = calcular_pedido(stock_actual, stock_minimo)
            if cantidad_pedir > 0:
                print(f"- {codigo} {nombre}: {cantidad_pedir} unidades a pedir")
        except Exception as e:
            print(f"Error en artículo {nombre}: {e}")

def agregar_unidades(inventario):
    try:
        codigo = int(input("Ingrese el código del producto: "))
        for code, nombre, stock_actual, stock_minimo in inventario:
            if code==codigo:
                print(f" El producto es: {nombre} ")

        unidades = int(input("Ingrese la cantidad de unidades a agregar: "))
        if unidades < 0:
            raise ValueError("La cantidad no puede ser negativa.")
        for articulo in inventario:
            if articulo[0] == codigo:
                articulo[2] += unidades
                print(f"Se agregaron {unidades} unidades a {articulo[1]}. Stock actual: {articulo[2]}")
        return
        print("Producto no encontrado.")
    except ValueError as ve:
        print(f"Error: {ve}")
    except Exception as e:
        print(f"Error inesperado: {e}")

def consultar_producto(inventario):
    try:
        codigo = int(input("Ingrese el código del producto: "))
        for articulo in inventario:
            if articulo[0] == codigo:
                print(f"\n=== INFORMACIÓN DEL PRODUCTO ===")
                print(f"Código: {articulo[0]}")
                print(f"Nombre: {articulo[1]}")
                print(f"Stock Actual: {articulo[2]}")
                print(f"Stock Mínimo: {articulo[3]}")
                return
        print("Producto no encontrado.")
    except ValueError:
        print("Error: el código debe ser un número entero.")
    except Exception as e:
        print(f"Error inesperado: {e}")

def menu_principal(inventario):
    while True:
        print("\n=== MENÚ PRINCIPAL ===")
        print("1. Consultar productos que necesitan reabastecerse")
        print("2. Agregar unidades a un producto")
        print("3. Consultar información de un producto por código")
        print("4. Salir")

```

```
try:
    opcion = int(input("Seleccione una opción: "))
except ValueError:
    print("Error: debe ingresar un número entre 1 y 4.")
    continue

if opcion == 1:
    consultar_reabastecimiento(inventario)
elif opcion == 2:
    agregar_unidades(inventario)
elif opcion == 3:
    consultar_producto(inventario)
elif opcion == 4:
    print("=== FIN DEL PROGRAMA ===")
    break
else:
    print("Opción inválida. Intente nuevamente.")
    continue

# Submenú para continuar o salir
while True:
    continuar = input("\n¿Desea regresar al menú principal (S/N)? ").strip().lower()
    if continuar == "s":
        break # vuelve al menú principal
    elif continuar == "n":
        print("=== FIN DEL PROGRAMA ===")
        return # termina la aplicación
    else:
        print("Opción inválida. Ingrese 'S' para regresar o 'N' para salir.")

# Inventario inicial
inventario = [
    [101, "Teclado", 3, 5],
    [102, "Mouse", 10, 8],
    [103, "Monitor", 2, 4],
    [104, "Impresora", 6, 6],
    [105, "USB", 1, 7]
]

# Ejecutar programa
menu_principal(inventario)
```

7. conclusión

La gestión de inventarios requiere soluciones confiables que integren validaciones y control de errores, garantizando la disponibilidad de productos y la eficiencia en los procesos. La programación estructurada aporta las bases para diseñar aplicaciones robustas, mientras que las referencias bibliográficas revisadas fortalecen la comprensión de operadores, funciones y modularidad en el desarrollo de software.

Asimismo, el uso de GitHub resulta especialmente relevante, ya que permite aplicar buenas prácticas de colaboración, control de versiones y documentación en proyectos de programación. Estas herramientas no solo facilitan el trabajo en equipo, sino que también aseguran la trazabilidad y la mejora continua del código.

En síntesis, el ejercicio contribuye al afianzamiento de habilidades técnicas y al reconocimiento de la importancia de integrar tanto fundamentos teóricos como herramientas modernas de gestión de proyectos en el ámbito de la programación.

8. Referencias

Moreno Pérez, J. C. (2014). Programación en lenguajes estructurados. RA-MA Editorial. (pp. 15-26).

Moreno Pérez, J. C. (2015). Programación. RA-MA Editorial. (pp. 21-29).

Ramírez Marín, J. H. (2019). Fundamentos iniciales de lógica de programación I. Algoritmos en PseInt y Python. Institución Universitaria de Envigado. (Capítulos 1 y 2).

Ceballos Sierra, F. J. (2015). *C/C++. Curso de programación* (4ª ed.). Elibro. Recuperado de: <https://elibro-net.bibliotecavirtual.unad.edu.co/es/ereader/unad/106454>

Deitel, P., & Deitel, H. (2016). *Cómo programar en C/C++*. Pearson Educación. Disponible en Biblioteca Virtual UNAD: <https://www-ebooks7-24-com.bibliotecavirtual.unad.edu.co/?il=16931&pg=145>

GitHub, Inc. (2025). *Acerca de GitHub y Git*. GitHub Docs. Recuperado de: <https://docs.github.com/es/get-started/start-your-journey/about-github-and-git>